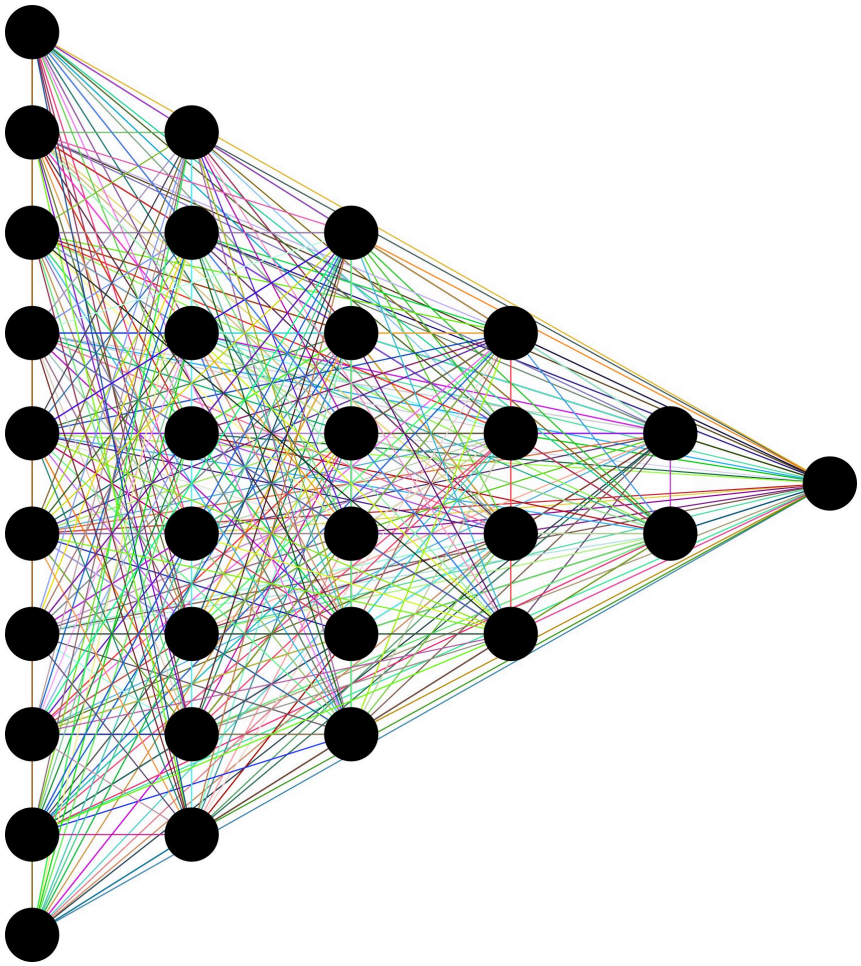
Università degli Studi di Verona

Intelligenza Artificiale

Appunti del corso 2026/2027



3.14

Anno Accademico 2026/2027

Indice

1 Che cos'è l'intelligenza artificiale 4
1.0.1 The imitation gaming 4
1.0.2 Artificial Intelligence 4
1.0.3 AI oggi 4
2 Che cos'è il machine learning 4
2.0.1 Tre famiglie principali 5
2.1 Agenti intelligenti 5
2.1.1 Come dovrebbe agire un Agente intelligente? 6
2.1.2 A popular framework: Markov Decision Process 6
2.1.2.1 Frozen Lake 6
3 Risoluzione dei problemi mediante ricerca 7
3.0.1 Esempio : Vacuum-cleaner world 7
3.0.2 Esempio di agent program 8
3.1 Razionalità 8
3.1.1 PEAS 9
3.1.2 Tipi di problemi 9
3.2 Problem solving agents 9
3.2.0.1 Esempio: Viaggio in Romania 10
3.2.0.2 Esempio: Vacuum world state soace graph 11
3.2.0.3 Esempio: The 8-Puzzle 12
3.2.0.4 Esempio: Three Search 12
3.3 Implementazione: Stati vs Nodi 13
3.3.1 Strategie di ricerca 14
4 Uninformed search strategies 15
4.1 Breadth-first search 15
4.1.1 Espandere il nodo meno profondo 15
4.1.2 Proprietà 16
4.2 Uniform cost search 16
4.2.1 Implementazione 16
4.2.2 Proprietà 16
4.2.3 Ottimalità di UCS 16
4.3 Depth-first search 17
4.3.1 Implementazione 17
4.3.1.1 Implementazione ricorsiva 18
4.3.2 Proprietà 18
4.3.3 Variante: Iterative deepening search 18
4.3.3.1 Proprietà 19
4.4 BSF vs IDS 19
4.5 Riassunto delle Proprietà 19
5 Informed search strategies 20
5.1 Best-First search 20
5.1.1 Greedy search 20
5.1.1.1 Greedy search esempio: 20
5.1.1.2 Proprietà: 21
5.1.2 A^* search 21
5.1.2.1 A^* search esempio: 21

5.1.2.2 Proprietà di A^* 22
5.2 Definizione di euristica 22
5.2.1 Euristica consistente 22
5.2.2 Euristica ammissibile vs Euristica consistente 22
5.2.3 Euristica ammissibile 23
5.2.4 Esempi tipici di ricerca: 24
5.2.5 Problemi rilassati 24
5.2.6 Funzioni euristiche e algoritmi di ricerca 24

1 Che cos'è l'intelligenza artificiale

- Nasce negli anni 50 con Alan Turing con il **Turing Test**
- 1996 **Deep Blue** per giocare a scacchi
- 2026 **Generative AI**

1.0.1 The imitation gaming

«Propongo di prendere in considerazione la domanda: «Le macchine possono pensare?». Si dovrebbe iniziare definendo il significato dei termini «macchina» e «pensare». Le definizioni potrebbero essere formulate in modo da riflettere, per quanto possibile, l'uso comune delle parole; tuttavia, questo atteggiamento è pericoloso. Se il significato delle parole «macchina» e «pensare» deve essere ricavato esaminando il modo in cui vengono comunemente usate, è difficile sottrarsi alla conclusione che.. «»

1.0.2 Artificial Intelligence

Nel 1956 fecero un progetto per capire se si potevano sviluppare dei sistemi per risolvere problemi più complessi in ambito umano e qui si usa per la prima volta il termine «Artificial Intelligence»

1.0.3 AI oggi

Robocup 2026



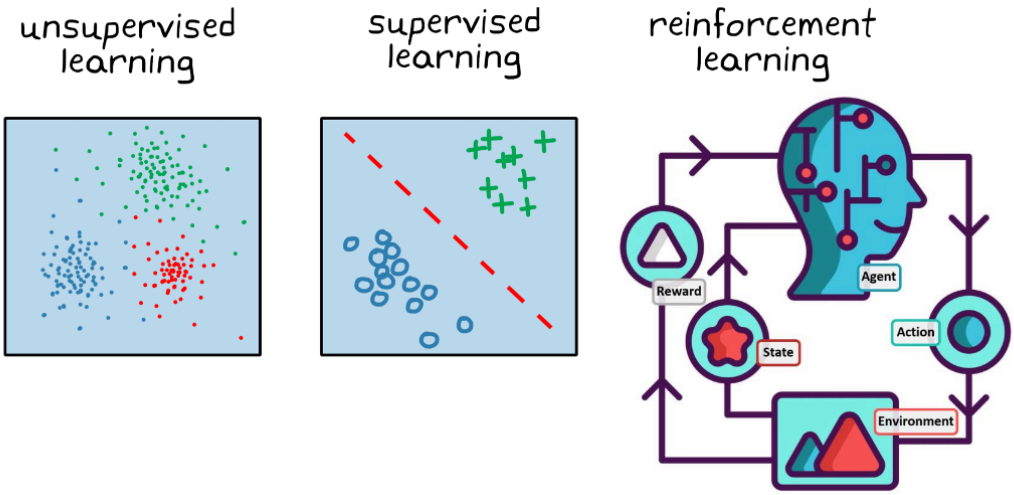
2 Che cos'è il machine learning

La programmazione permette ai calcolatori di risolvere task, mentre il machine learning insegna ai calcolatori come risolvere gli stessi medesimi problemi ma in modo autonomo.

Perche il machine learning è tanto usato?

Grazie alla raccolta dei dati negli ultimi anni usare tecniche di ML per risolvere problemi è diventato molto più facile.

2.0.1 Tre famiglie principali



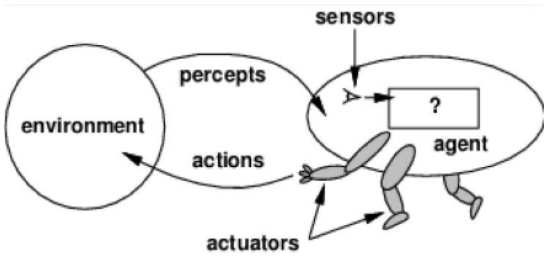
- **Non supervisionato** : non diamo informazioni al calcolatore se non solo la rappresentazione dei dati, la macchina decide in che modo suddividere i gruppi
- **Supervisionato** : diamo informazioni al calcolatore definendo noi in che modo discriminare i dati
- **Reinforcment learning** : un agente compie azioni nel mondo con un meccanismo di riccompense

2.1 Agenti intelligenti

Nella comunità che studia i sistemi ad agenti, è sorprendentemente difficile rispondere alla domanda **"che cos'è un agente?"**, proprio come nell'IA è difficile definire in modo universalmente accettato "che cos'è l'intelligenza?".

Il termine agent viene usato molto, anche da ricercatori di aree molto vicine, ma non esiste una definizione unica condivisa. Questo non è necessariamente un problema, perché si possono comunque costruire applicazioni utili anche senza accordarsi perfettamente sulla terminologia.

Però c'è il rischio che agent diventi una parola vaga, usata in modo improprio e quindi fonte di confusione. Per questo gli autori ritengono utile discutere il significato del termine e distinguono due modi principali di intendere "agente": uno debole, relativamente poco controverso, e uno più forte, più discutibile.



Russel e Norvig: AI a modern approach

Un agente è dotato di 3 caratteristiche:

- Analizza l'ambiente

- Agisce nell'ambiente
- Ha sempre un obiettivo

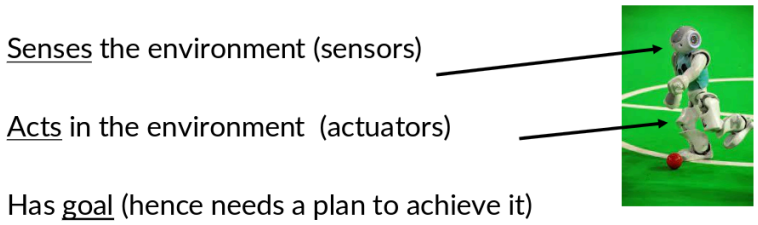
2.1.1 Come dovrebbe agire un Agente intelligente?

1) Misurazione delle performance

L'agente valuta se l'ambiente è in grado di far raggiungere all'agente l'obiettivo

2) Razionalità

L'agente sceglie sempre di massimizzare il valore che lo porta al raggiungimento dell'obiettivo



2.1.2 A popular framework: Markov Decision Process

Un esempio classico di agente è **Markov Decision Process** dove abbiamo:

- **state**: Rappresentazione dell'ambiente
- **action**: Come l'agente può modificare l'ambiente
- **transition model**: Gli effetti che hanno le azione sull'ambiente

$$T : \text{state, action} \rightarrow \text{next state}$$

- **reward**: Ricompensa immediata in risposta alla azione

$$R : \text{state, action,(next state} \rightarrow \text{real number}$$

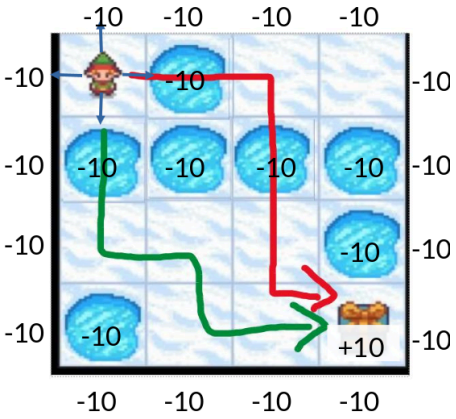
- **policy**: scelta delle azioni per massimare la ricompensa nel lungo termine

$$\pi : (\text{state}) \rightarrow \text{action}$$

2.1.2.1 Frozen Lake



Prendimao in considerazione gli elementi principali della catena di Markov:



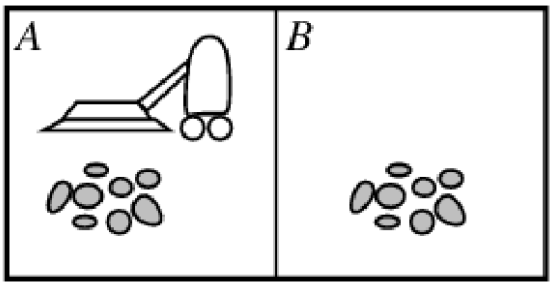
3 Risoluzione dei problemi mediante ricerca

Con il termine **agente** ci possiamo riferire a: umani, robot, softbot, termostati ecc..
L' **agent funtion** associa a ogni possibile sequenza di percezioni un'azione da eseguire:

$$f : P^* \rightarrow A$$

L'**agent program** invece viene eseguito sul calcolatore per produrre f

3.0.1 Esempio : Vacuum-cleaner world



- Perceptions: Posizione nello spazione e il contenuto [A, Dirty]
- Actions: left, right, Suck, NoOp

Percept sequence	Action
[*A*, *Clean*]	*Right*
[*A*, *Dirty*]	*Suck*
[*B*, *Clean*]	*Left*
[*B*, *Dirty*]	*Suck*
[*A*, *Clean*], [*A*, *Clean*]	*Right*
[*A*, *Clean*], [*A*, *Dirty*]	*Suck*
⋮	⋮

Domanda

se un agent ha $|P|$ possibili perceptions, quante entries avrà l'agent function dopo T passi?

Risposta

$$\sum_{t=1}^{T} |P|^t$$

3.0.2 Esempio di agent program

function **Reflex-Vacuum-Agent([location, status])** return an action:

```
if status = Dirty then return Suck
else if location = A then return Right
else if location = B then return Left
```

3.1 Razionalità

La misura di prestazione fissa valuta la sequenza dell'ambiente.

- è possibile pulire ogni punto dell'area in tempo T?
- un punto per ogni casella pulita ad ogni passo T, meno un punto per ogni mossa?
- penalizzare per $>$ K caselle sporche?

Un **rational agent** sceglierà qualsiasi azione che massimizzi il valore atteso della performance data la sequenza di percezioni acquisita fino a quel momento

Ricordiamo però che:

Rational $\neq$ omniscent

- percezioni possono non fornire tutte le informazioni rilevanti

Rational $\neq$ clairvoyant

- le azioni compiute possono non essere quelle che ci saremmo aspettati

Rational $\neq$ successful

- poichè rational $\Rightarrow$ exploration, leraning, autonomy

Esercizio

Considera il seguente ambiente:

- Ci sono **tre stanze (A, B, C)** e **due robot (r1, r2)**.
- **r1** può pattugliare le stanze **A e B**.
- **r2** può pattugliare le stanze **B e C**.
- **r1** parte dalla stanza **A**, mentre **r2** parte dalla stanza **C**.
- Il **tempo di spostamento tra le stanze è pari a zero**.

Misura di prestazione: minimizzare la somma dell'**idleness media** di tutte le stanze.

Idleness media = somma degli intervalli di tempo durante i quali la stanza **non viene visitata da nessun robot** / intervallo di tempo totale.

Quale sarebbe un **comportamento razionale** per questo ambiente?

3.1.1 PEAS

Per progettare un **rational agent** dobbiamo specificare il **task environment**.
Considerando un esempio: the task of designing an automated taxi:

- **performance measure** → safety, destination, profits, legality, comfort …
- **environment** → city street/freeways, traffic, pedestrains, weather …
- **actuators** → steering, accelerator, brake, horn, speaker/display …
- **sensors** → video, accelerometers, gauges, engine sensors, keyboard, GPS …

Il tipo di ambiente determina in larga misura la progettazione dell'agente

	Crossword	Robo-selector	Poker	Taxi
Observable??	Yes	Partly	Partly	Partly
Deterministic??	Yes	No	No	No
Episodic??	No	Yes	No	No
Static??	Yes	No	Yes	No
Discrete??	Yes	No	Yes	No
Single-agent??	Yes	Yes	No	No

Poichè il mondo reale è ovviamente parizalmente osservabile, stocastico, sequenziale, dinamico, continuo e multi agentico

3.1.2 Tipi di problemi

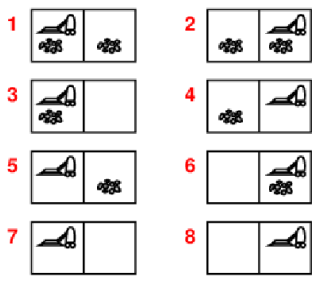
- **Deterministico e completamente osservabile → problema a stato singolo**
  - L'agente sa esattamente in quale stato si troverà.
  - La soluzione è una **sequenza di azioni**.
- **Non osservabile → problema conformante**
  - L'agente può non avere alcuna informazione sullo stato in cui si trova.
  - La soluzione, se esiste, è una **sequenza di azioni**.
- **Non deterministico e/o parzialmente osservabile → problema di contingenza**
  - Le percezioni forniscono nuove informazioni sullo stato corrente.
  - La soluzione è un **piano contingente** oppure una **policy**.
  - Spesso ricerca ed esecuzione vengono alternate.
- **Spazio degli stati sconosciuto → problema di esplorazione** (**online**)

Esempio: Vacuum world

Single-state, start in #5. Solution??
[*Right*, *Suck*]

Conformant
start in $\{1, 2, 3, 4, 5, 6, 7, 8\}$
e.g., *Right* goes to $\{2, 4, 6, 8\}$. Solution??
[*Right*, *Suck*, *Left*, *Suck*]

Contingency, start in #5
Murphy's Law: *Suck* can dirty a clean carpet
Local sensing: dirt, location only.
Solution??
[*Right*, if *dirt* then *Suck*]



3.2 Problem solving agents

Derivazione di un agente generale: **agenti basati su obiettivi**

- formulano un obiettivo e un problema dato uno stato corrente

- cercano una soluzione
- eseguono la soluzione **ingnorando** le percezioni

Questo a volte viene anche chiamato: **offline problem solving** → solutions executed «eyes closed»

```
function Simple-Problem-Solving-Agent(percept) returns an action
    static: seq, an action sequence, initially empty
            state, some description of the current world state
            goal, a goal, initially null
            problem, a problem formulation

    state ← Update-State(state, percept)

    if seq is empty then
        goal ← Formulate-Goal(state)
        problem ← Formulate-Problem(state, goal)
        seq ← Search(problem)

    action ← First(seq)
    seq ← Rest(seq)

    return action
```

3.2.0.1 Esempio: Viaggio in Romania

Esempio

On holiday in Romania; currently in Arad.
Flight leaves tomorrow from Bucharest

Formulate goal

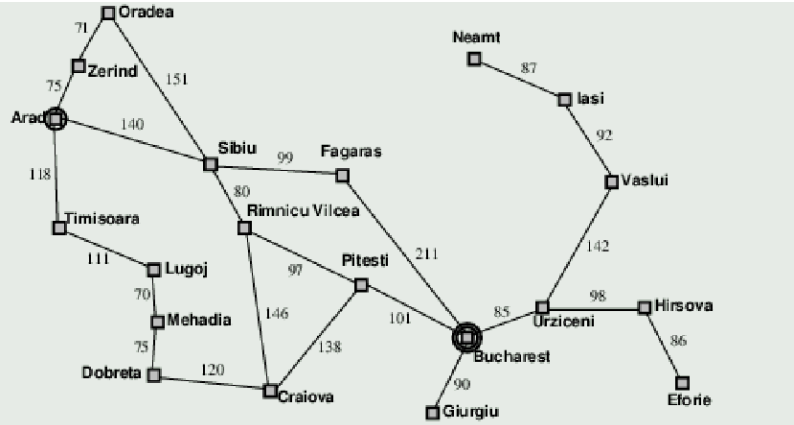
- be in Bucharest

Formulate problem

- **states**: various cities
- **actions**: drive between cities

Find Solution:

- sequence of cities e.g: Arad, Sibiu, Fagaras, Bucharest



Un problema è definito da quattro elementi:

- stato iniziale
  - ad esempio, «ad Arad»
- funzione successore $S(x)$ = insieme di coppie azione-stato
  - ad esempio, $S(A) = \{< \text{Arad} \rightarrow \text{Zerind}, \text{Zerind} >, ...\}$
- test dell'obiettivo, può essere
  - esplicito, ad esempio, $x =$ a Bucharest
  - implicito, ad esempio, NoDirt(x)
- costo del percorso (additivo)
  - ad esempio, somma delle distanze, numero di azioni eseguite, ecc.

$c(x, a, y)$ è il costo del passo, assunto essere ≥ 0

Una soluzione è una sequenza di azioni che porta dallo stato iniziale a uno stato obiettivo.

Il mondo reale è assurdamente complesso poichè lo spazio degli stati deve essere astratto per risolvere il problema

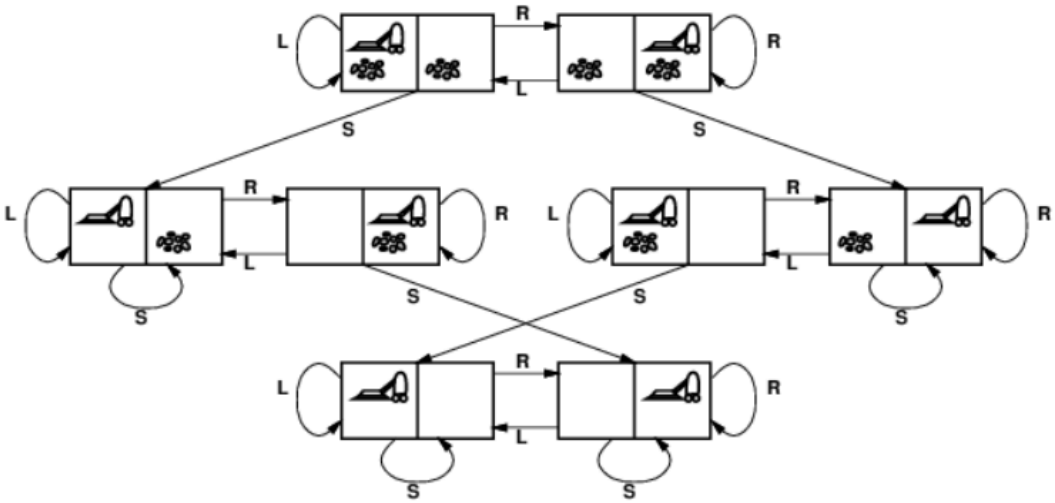
- stato (astratto) = insieme di stati reali
- azione (astratta) = combinazione complessa di azioni reali

ad esempio, «Arad -> Zerind» rappresenta un insieme complesso di possibili percorsi, deviazioni, soste, ecc.
Per garantire la realizzabilità, qualsiasi stato reale «ad Arad» deve poter raggiungere qualche stato reale «a Zerind»

- soluzione (astratta) = insieme di percorsi reali che sono soluzioni nel mondo reale

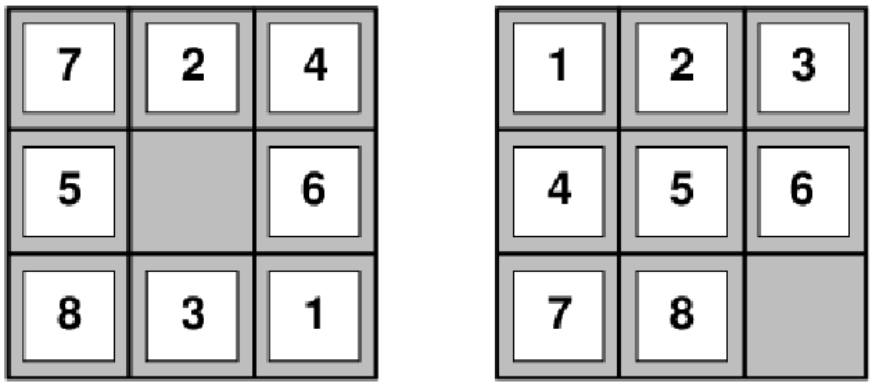
Ogni azione astratta dovrebbe essere «più semplice» del problema originale!

3.2.0.2 Esempio: Vacuum world state soace graph



- **states** = screte dirt and robot location (ingore dirt amounts)
- **actions** = Left, Right, Suck, NoOp
- **goal test** = no dirt
- **path cost** = 1 per action (0 for NoOp)

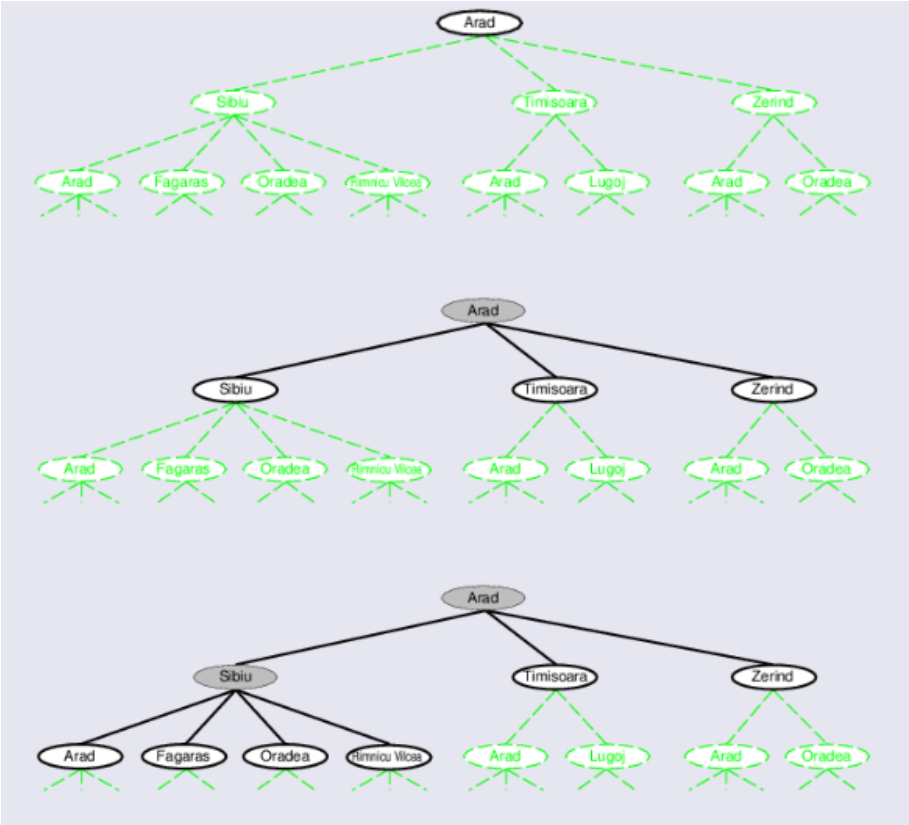
3.2.0.3 Esempio: The 8-Puzzle



- **states** = integer locations of tiles (ignore intermediate potisions)
- **actions** = move blank left, right, up, down
- **goal test** = given goal state
- **path cost** = 1 per move

[Nota: soluzioni ottimali di n-Puzzle $\in$ NP-hard]

3.2.0.4 Esempio: Three Search

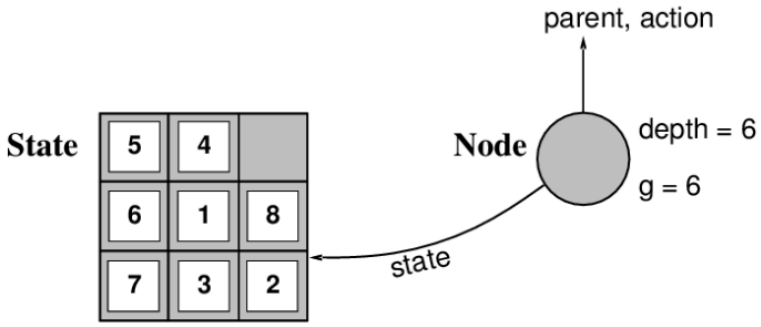


3.3 Implementazione: Stati vs Nodi

Uno **stato** è la rappresentazione di una configurazione fisica

Un **nodo** invece è una struttura dati composta da parti di uno search tree

- parent
- action
- children
- depth
- path cost (i.e, g(x))



La funzione **expand** crea nuovi nodi, usano il **SuccessorFn** del problema per creare il corrispondente stato

Tree Search

```
function Tree-Search(problem, frontier) returns a solution, or failure
frontier <- Insert(Make-Node(problem.Initial-State))
while not IsEmpty(frontier) do
  node <- Pop(frontier)
  if problem.Goal-Test(node.State) then
    return node
  frontier <- InsertAll(Expand(node, problem))
end loop
return failure
```

Expand

```
function Expand(node, problem) returns a set of nodes
successors <- the empty set
for each action, result in Successor-Fn(problem, node.State) do
  s <- a new Node

  s.Parent-Node <- node
  s.Action <- action
  s.State <- result
  s.Path-Cost <- node.Path-Cost + Step-Cost(node.State, action, result)
  s.Depth <- node.Depth + 1

  add s to successors
return successors
```

3.3.1 Strategie di ricerca

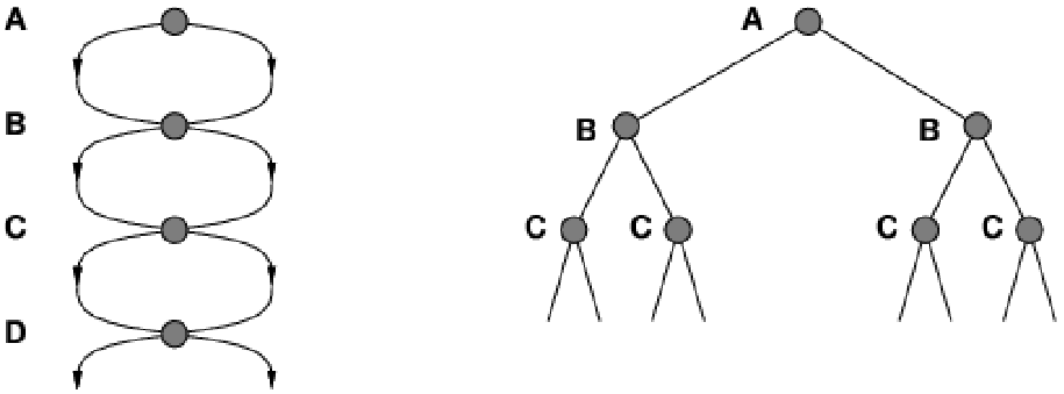
Una strategia è definita scegliendo l'ordine di espansione dei nodi. Le strategie vengono valutate secondo le seguenti dimensioni:

- **completezza** — permette di trovare una soluzione, se ne esiste una?
- **complessità temporale** — numero di nodi generati/espansi
- **complessità spaziale** — numero massimo di nodi mantenuti in memoria
- **ottimalità** — permette di trovare una soluzione di costo minimo?

La complessità temporale e spaziale vengono misurate in funzione di:

- b — fattore di ramificazione massimo dell'albero di ricerca
- d — profondità della soluzione di costo minimo
- m — profondità massima dello spazio degli stati (può essere ∞)

Fallire nel rilevare stati ripetuti può trasformare il problema da lineare a non lineare!



Definiamo allora il problema come:

```
function Graph-Search(problem, frontier) returns a solution, or failure
explored <- an empty set
frontier <- Insert(Make-Node(problem.Initial-State))

while not IsEmpty(frontier) do
  node <- Pop(frontier)
  if problem.Goal-Test(node.State) then
    return node
  if node.State is not in explored then
    add node.State to explored
    frontier <- InsertAll(Expand(node, problem))
  end if
end loop

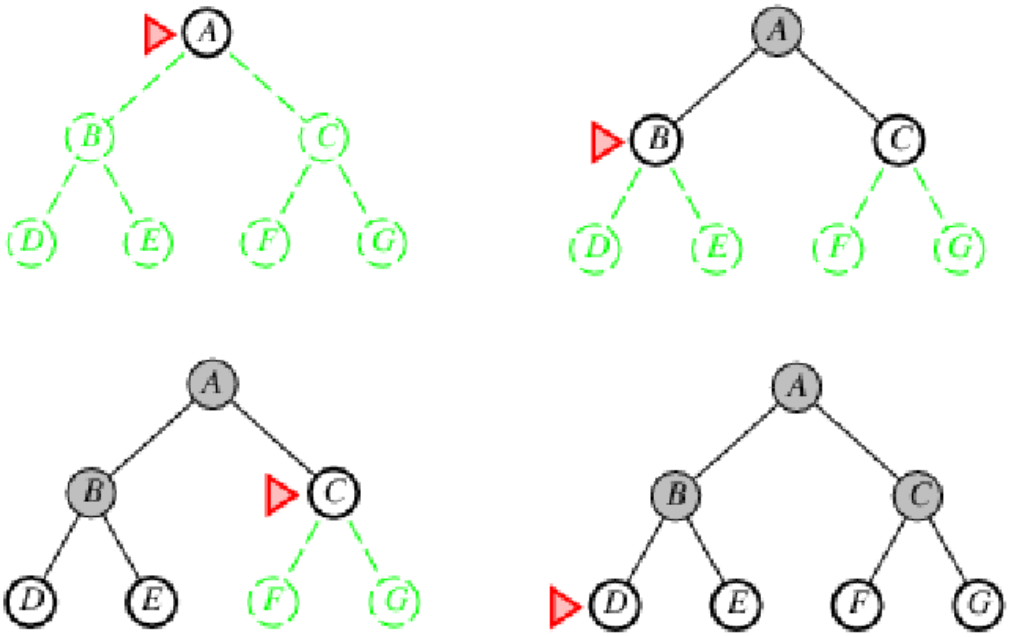
return failure
```

4 Uninformed search strategies

4.1 Breadth-first search

4.1.1 Espandere il nodo meno profondo

La **frontier** ovvero la frontiera (nodi che non sono ancora stati scoperti) sono rappresentati come code FIFO, il nuovo sucessore viene messo alla fine.




```
function BFS(problem) returns a solution, or failure
    node ← node with State=problem.Initial-State, Path-Cost=0
    if problem.Goal-Test(node.State) then return node
    explored ← empty set
    frontier ← FIFO queue with node as the only element

    loop do
        if frontier is empty then return failure
        node ← Pop(frontier)
        add node.State to explored

        for each action in problem.Actions(node.State) do
            child ← Child-Node(problem, node, action)

            if child.State is not in explored or frontier then
                if problem.Goal-Test(child.State) then return child
                frontier ← Insert(child)
            end if
        end for
    end loop
```

4.1.2 Proprietà

Completo? si. se b è finito (numero nodi)
Tempo? $b + b^2 + b^3 + ... + b^d = \mathrm{O}(b^d)$
spazio? $\mathrm{O}(b^d)$ (ogni nodo in memoria)
ottimale? si se il costo rimane costante ad ogni passagio (in generale non lo è)

Spazio

E' il problema principale di BFS, poiche può generare nodi a 100MB/sec in 24h può raggiungere 8640GB

4.2 Uniform cost search

Espandere il nodo non espanso a costo minimo (i.e minimum step cost), una versione di Dijkstra specializzata in ricerca

4.2.1 Implementazione

frontier = coda ordinata per path cost, dal più basso al più alto (equivalente a BFS se il costo ad ogni step è costante)

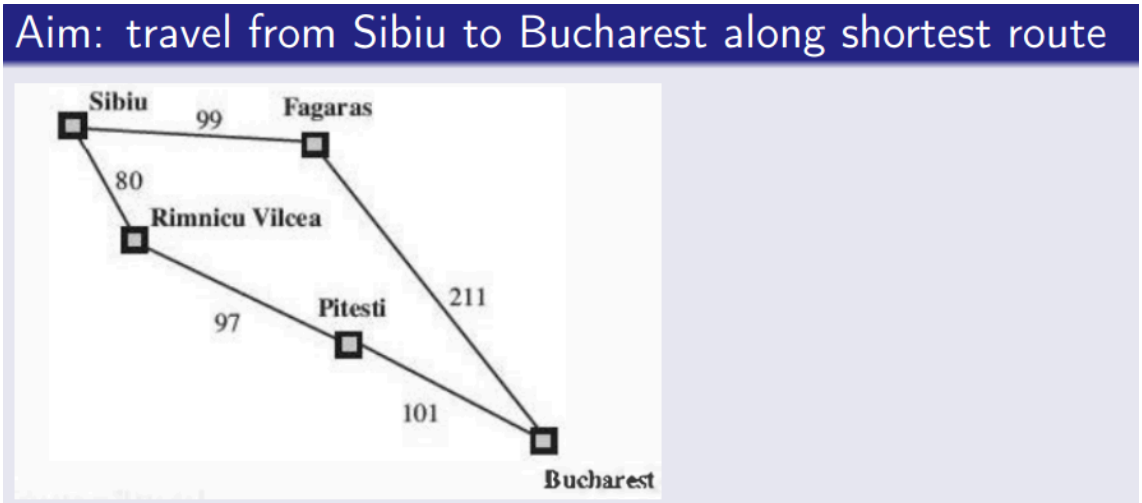
4.2.2 Proprietà

Completo? si. se il costo ad ogni step è $> \varepsilon$
Tempo? # di nodi con $g \leq$ del costo ottimale, $O\left(b^{1+\left\lfloor \frac{C^*}{\varepsilon} \right\rfloor}\right)$ dove $C*$ è il costo ottimale
spazio? # di nodi con $g \leq$ del costo ottimale, $O\left(b^{1+\left\lfloor \frac{C^*}{\varepsilon} \right\rfloor}\right)$ dove $C*$ è il costo ottimale
ottimale? si, i nodi si espandono in ordine di $g(n)$

due modifiche a BFS per ottener una UCS

1) Goal test quando espandi, non quando generi
2) Sostituire un nodo nella frontier se trovi un percorso meno costoso

4.2.3 Ottimalità di UCS

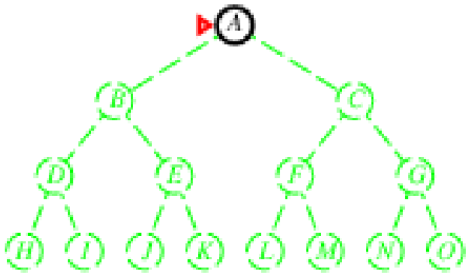


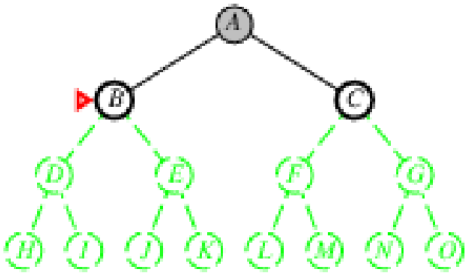
Consideriamo la mappa in figura e UCS, ci dimostra che per l'appunto:
1) Goal test quando espandi, non quando generi
2) Sostituire un nodo nella frontier se trovi un percorso meno costoso

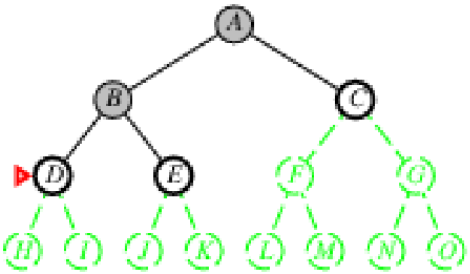
4.3 Depth-first search

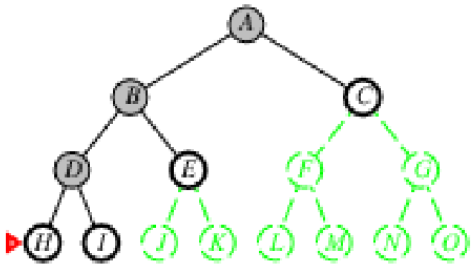
4.3.1 Implementazione

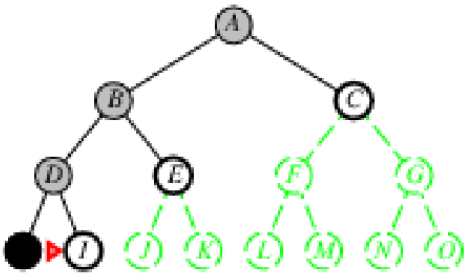
frontier = coda LIFO

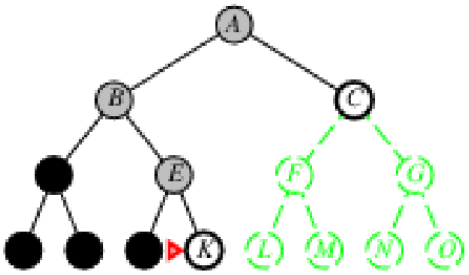


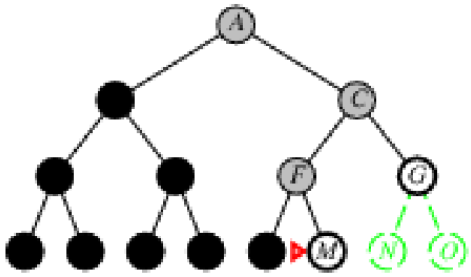












4.3.1.1 Implementazione ricorsiva

```
function DLS(problem, limit) returns soln/fail/cutoff
    R-DLS(Make-Node(problem.Initial-State), problem, limit)

function R-DLS(node, problem, limit) returns soln/fail/cutoff
    if problem.Goal-Test(node.State) then return node
    else if limit = 0 then return cutoff
    else
        cutoff-occurred? ← false

        for each action in problem.Actions(node.State) do
            child ← Child-Node(problem, node, action)
            result ← R-DLS(child, problem, limit-1)

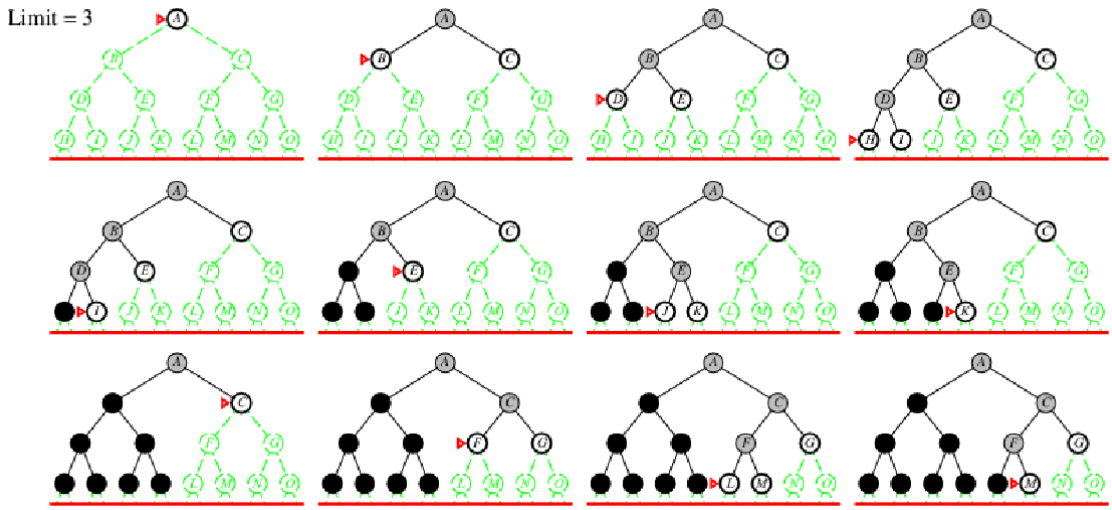
            if result = cutoff then cutoff-occurred? ← true
            else if result ≠ failure then return result
        end for

        if cutoff-occurred? then return cutoff
        else return failure
    end else
```

4.3.2 Proprietà

Completo? no, fallisce per spazzi con profondità infinit e in presenza di cappi. Modifiche al grafo per evitare stati ripetuti portano a rendere il sistema completo
Tempo? O(b^m): terribile se m è più grande di d
spazio? O(bm) spazio lineare
ottimale? No!

4.3.3 Variante: Iterative deepening search




```
function IDS(problem) returns a solution
    inputs: problem, a problem

    for depth ← 0 to ∞ do
        result ← DLS(problem, depth)
        if result ≠ cutoff then return result
    end
```

4.3.3.1 Proprietà

Completo? si
Tempo? $db^1 + (d-1)b^2 + ... + b^d = \mathrm{O}(b^d)$
spazio? $\mathrm{O}(b^d)$
ottimale? si, se il costo ad ogni step è = 1

4.4 BSF vs IDS

Scegliamo b = 10 e d = 5, avremmo che :

$$N(\text{IDS}) = 6 + 50 + 400 + ... = 123456$$

$$N(\text{BFS}) = 10 + 100 + 1000 + ... = 111101$$

IDS ripete alcuni nodi ma il costo non diventa tanto più grande di BFS poichè la complessità è dettata dalla crescita esponenziale dei nodi

4.5 Riassunto delle Proprietà

Basandoci su tree-search

Criterion	BF	UC	DF	DL	ID
Complete?	Yes*	Yes*,†	No	Yes*, if $l \geq d$	Yes*
Time	b^d	$b^{1+\lfloor C^*/\epsilon \rfloor}$	b^m	b^l	b^d
Space	b^d	$b^{1+\lfloor C^*/\epsilon \rfloor}$	bm	bl	bd
Optimal?	Yes⋆	Yes	No	Yes⋆, if $l \geq d$	Yes⋆

*: complete if branching factor is finite
†: complete if step cost is $\geq \epsilon$
⋆: optimal if step costs are all identical

5 Informed search strategies

Una strategia è definita dalla scelta dell'ordine di espansione dei nodi

5.1 Best-First search

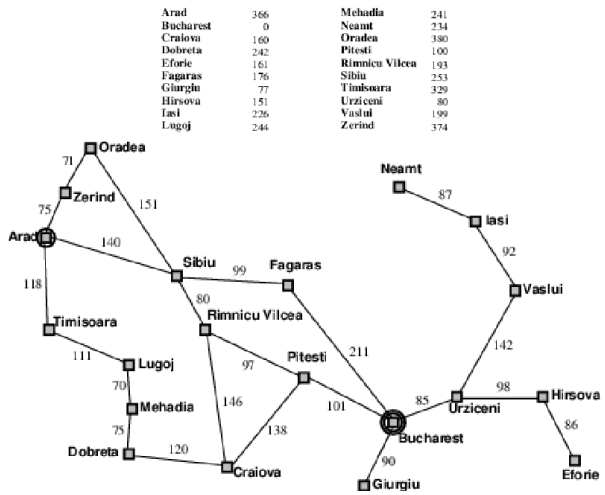
Idea = usare una funzione di valutazione per ogni nodo (stima di desiderabilità) per espandere il nodo non espanso più promettente (desidedrato)

Implementazione = la **frontier** è una coda ordinata descrescente in base alla desiderabilità

Casi speciali :

- greedy best-first search
- A^* search

Roamania with straight-line distances to Bucharest



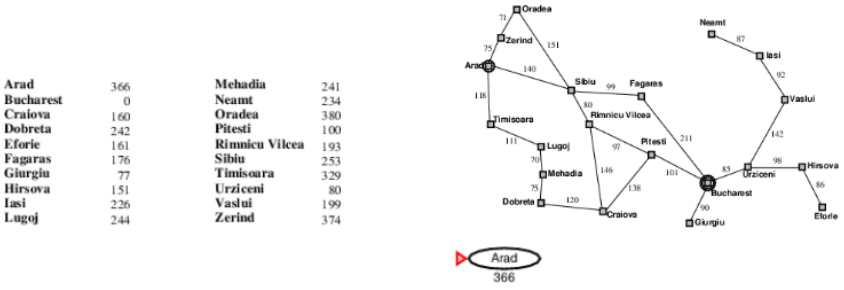
5.1.1 Greedy search

Sia una funzione di valutazione $h(n)$ euristica t.c Stima del costo da n all'obiettivo più vicino.

Esempio: $h_{SLD}(n)$ = straight-line distance da n a Bucharest.

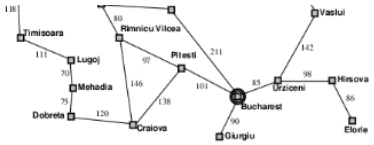
Greedy search espande il nodo che si avvicina di più all'obiettivo

5.1.1.1 Greedy search esempio:



Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244

Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374







5.1.1.2 Proprietà:

Completo? No, può bloccarsi in loop (completo in spazi finiti e con check su cicli)
Tempo? O(b^m) ma una buona euristica può migliorare mostrosamente
spazio? O(b^m) (ogni nodo in memoria)
ottimale? No

5.1.2 A^* search

Idea = evitare di espandere path che hanno un costo elevato.
Sia una funzione di valutazione $f(n) = g(n) + h(n)$ con:

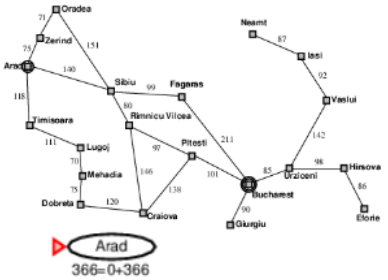
- $g(n)$ = costo per raggiungere nodo N
- $h(n)$ = stima del costo dall'obiettivo ad N
- $f(n)$ = stima del costo totale da N all'obiettivo

A^* usa una ottima euristica:
Sia $h(n) \leq h^*(n)$ dove $h^*(n)$ è il vero costo da n (richiede che $h(n) > 0$, quindi $h(G) = 0$ per ogni obiettivo G)

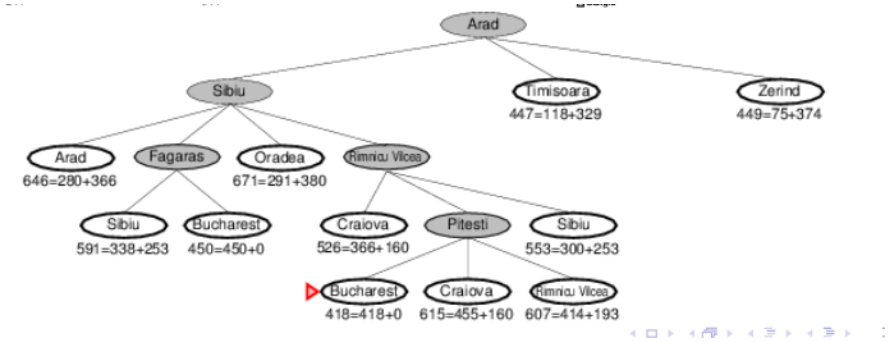
5.1.2.1 A^* search esempio:

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244

Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374







5.1.2.2 Proprietà di A^*

Completo? Si, fino a quando non ci sono infiniti doci con $f < f(g)$
Tempo? Esponenziale
spazio? Ogni nodo in memoria
ottimale? Si, **ma** richiede assunzioni euristiche (ammissibilità e consistenza) e search strategy (tree o graph search)

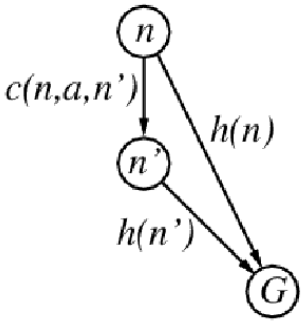
5.2 Definizione di euristica

5.2.1 Euristica consistente

Una euristica si può definire consistente se:

$$h(n) \leq c(n, a, n') + h(n')$$

1) Possiamo dimostrare che se h è consistente allora $f(n)$ non è decrescente su tutto il path
2) A^* espande nodi in ordine di crescita rispetto ad f, di conseguenza, troverà la soluzione a costo minimo.



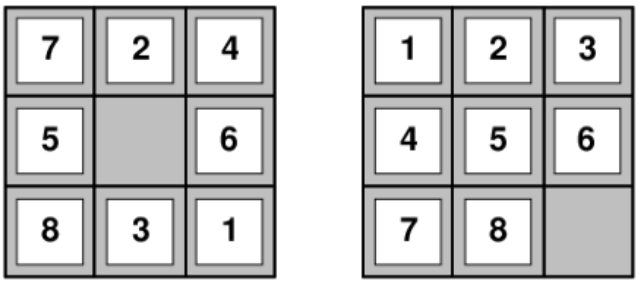
5.2.2 Euristica ammissibile vs Euristica consistente

Consistenza → Ammissibile
si può dimostrare per induzione sul percorso verso l'obiettivo.

Ammissibile $\not\to$ Consistenza
basti trovare un controesempio...

Tree-Search + euristica ammissibile $\to$ ottimale per A^*
Graph-Search + euristica ammissibile $\not\to$ ottimale per A^*

È possibile scartare il percorso ottimale verso un nodo già visitato, quindi:
Graph-Search + euristica consistente $\to$ ottimale per A^*

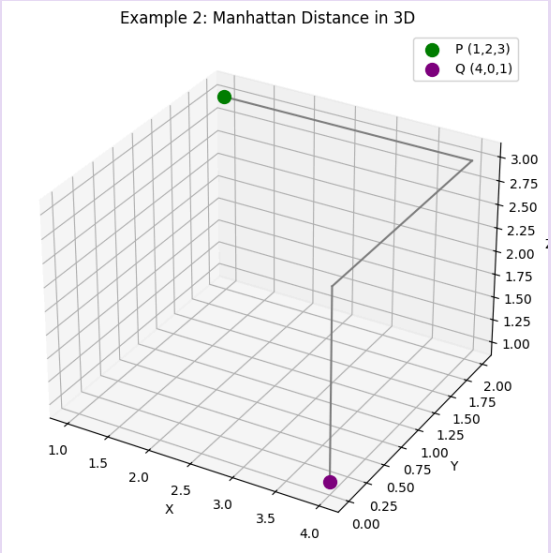


5.2.3 Euristica ammissibile

Prendiamo il problema del 8-puzzle:
$h_1(n)$ = numero di tessere fuori posto $h_2(n)$ = distanza di Manhattan totale

Distanza di Manhattan

è una metrica che misura la distanza tra due punti sommando il valore assoluto delle differenze delle loro coordinate



$h_1(S) = 6$
$h_2(n) = 4 + 0 + 3 + 3 + 1 + 0 + 2 + 1 = 14$

Se $h_2(n) \geq h_1(n)$ per ogni n, allora h_2 domina h_1 ed è migliore per la ricerca

5.2.4 Esempi tipici di ricerca:

Per d = 14:
IDS = 3473941 nodi
$A^*(h_1) = 539$ nodi
$A^*(h_2) = 113$ nodi

Per d = 24:
IDS ≈ 54000000000 nodi
$A^*(h_1) = 39135$ nodi
$A^*(h_2) = 1641$ nodi
Data una euristica ammissibile h_a e h_b : $h(n) = \max\left(h_{a(n)}, h_{b(n)}\right)$

Possiamo quindi dire:

- Ammissibile: non sovrastima mai il costo reale minimo per raggiungere l'obiettivo.
- Consistente: rispetta una particolare relazione tra il costo stimato di due nodi vicini.

5.2.5 Problemi rilassati

Euristiche ammissibili possono essere ricavate dalla esatta soluzione del costo della **versione rilassata** del problema, ovvero un problema al quale sono stati eliminati alcuni vincoli rendendo il problema più semplice da risolvere

- Se le regole di 8-puzzle sono rilassate quindi una casella può essere mossa **ovunque**, allora $h_1(n)$ ci da la soluzione più rapida
- Se le regole di 8-puzzle sono rilassate quindi una casella può essere mossa in una qualsiasi casella adiacente, allora $h_2(n)$ ci da la soluzione più rapida

Key point

Il costo della soluzione ottimale di un **problema rilassato** non è più grande del costo della soluzione ottimale del **problema reale**

5.2.6 Funzioni euristiche e algoritmi di ricerca

- Le **funzioni euristiche** stimano il costo del percorso più breve per raggiungere l'obiettivo.
- Buone euristiche possono ridurre drasticamente il costo della ricerca.
- **Greedy Best-First Search**:
  - Espande il nodo con il valore $h(n)$ più basso.
  - Non è completo in generale e non garantisce una soluzione ottima.
- **A* Search**:
  - Espande il nodo con il valore $f(n) = g(n) + h(n)$ più basso.
  - È completo e ottimale sotto opportune condizioni.
  - È anche **ottimamente efficiente** (a meno dei casi di parità nella scelta dei nodi, per la ricerca in avanti).
- Le **euristiche ammissibili** possono essere ricavate dalla soluzione esatta di problemi rilassati.